

Contents

Disclaimer	1
Getting Started	1
Lesson 1: Your First Program	1
How Do We Display a Message in Python?	2
Important Notice	2
Python as a Language of Mathematics	3
1. Numbers → Python Numbers	3
Integers, Real Numbers, and Complex Numbers	3
2. Arithmetic → Python Arithmetic	6
Addition	7
Subtraction	7
Multiplication	7
Division	7
2.1 Exponentiation → Python **	8
2.2 Floor Division and Remainder → Python Arithmetic	9
Floor Division	9
Remainder	9
2.3 Order of Operations → Python	10
Using Parentheses	11
A Mathematical Expression in Python	11
Summary: Mathematics → Python Arithmetic	12

This course, **Python as a Language of Mathematics**, assumes that you already think mathematically.

Disclaimer

Before we begin, I want to be upfront about what this course is — and what it isn't.

This is **not** a general-purpose “learn to code” course. We won't be building apps, websites, or games. Instead, we're learning Python as a **language of mathematics** — a tool for expressing and solving mathematical ideas.

So whenever we introduce a Python concept, I'll connect it directly to a mathematical idea, rather than teaching it in isolation the way a typical programming tutorial would. This course also assumes you already think mathematically — we're not teaching math itself here, just how to express it in Python.

Getting Started

If you're new to Python, you don't need to install Jupyter Notebook or Visual Studio Code to get started — especially if you have internet access. Several online platforms let you write and run Python code directly in your browser, such as **Google Colab** and **Kaggle**.

In this course, we'll be using **Google Colab**.

To get started: open your web browser, search for “Google Colab,” and click the link. This takes you to the environment where we'll write and run all our Python code throughout this course.

Let's get started!

Lesson 1: Your First Program

You can actually start coding right here, just as you would start working with mathematics.

We can begin by typing numbers, performing basic arithmetic, and evaluating mathematical expressions.

For example, we can start with something as simple as:

$$2 + 3$$

Let's see how Python handles this.

```
2 + 3
```

```
5
```

That is simple, right?

However, there is a particular way we display information in Python.

How Do We Display a Message in Python?

To display something on the screen, we use a function called `print()`.

Notice that we write it as:

```
print()
```

It is **not**:

```
Print()
```

and it is **not**:

```
PRINT()
```

It must be written as:

```
print()
```

Important Notice

Here is the first important lesson you are learning about Python:

Python is a case-sensitive language.

This means that Python treats uppercase and lowercase letters as different.

Let's see what happens when we try these three examples.

```
print(2 + 3)
```

```
Print(2 + 3)
```

```
PRINT(2 + 3)
```

The first one works because `print()` is the correct name of the Python function.

The other two will produce an error because `Print()` and `PRINT()` are different from `print()`.

So, remember:

In Python, uppercase and lowercase letters matter.

Another example is displaying **text** in Python.

Suppose we want to display the following message on the screen:

Welcome to Python as a Language of Mathematics.

This is not a number or a mathematical expression. It is a statement, or text.

So, how do we tell Python that what we have written is text?

We need to enclose the text in **quotation marks**. We can use either single quotation marks:

```
'Welcome to Python as a Language of Mathematics.'
```

or double quotation marks:

```
"Welcome to Python as a Language of Mathematics."
```

And, of course, if we want Python to display the message on the screen, we use the `print()` function:

```
print('Welcome to Python as a Language of Mathematics.')
```

We can also write:

```
print("Welcome to Python as a Language of Mathematics.")
```

Both will give us the same output.

So, remember: when we want Python to treat something as text, we enclose it in quotation marks.

Python as a Language of Mathematics

1. Numbers → Python Numbers

Basically, everything in Python is an object. Numbers are objects that we can use in mathematical operations. Understanding how Python represents different types of numbers is important because, as we move through this course, we will use numbers constantly in our mathematical calculations.

Integers, Real Numbers, and Complex Numbers

i. Integers In mathematics, the integers are represented by

$$\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}.$$

Integers are whole numbers. They do not have decimal or fractional parts. They include negative numbers, zero, and positive numbers.

For example,

$$-5, \quad -2, \quad 0, \quad 3, \quad 10$$

are all integers.

Python also has integers. We can write them directly, without using quotation marks.

For example:

```
-5
```

```
0
```

```
10
```

Python recognizes these as integers.

We can also perform mathematical operations with integers. For example,

```
2 + 3
```

gives

```
5
```

Similarly,

```
10 - 7
```

gives

```
3
```

and

```
4 * 5
```

gives

```
20
```

So, when we write an integer such as 5 in Python, Python recognizes it as a number that can be used in mathematical operations.

This is our first connection between mathematics and Python:

$$5 \rightarrow 5$$

The mathematical number (5) is represented in Python by 5.

The same idea applies to other integers:

$$-3 \rightarrow -3$$

$$0 \rightarrow 0$$

$$12 \rightarrow 12.$$

So Python does not require us to do anything special to represent an integer. We simply write the integer.

In mathematics, we call these numbers integers. In Python, integers belong to a data type called `int`. So when you see `int` in Python, think about the mathematical set of integers, $\mathbb{Z}$.

ii. Real Numbers In mathematics, the set of real numbers is represented by

$$\mathbb{R}.$$

Real numbers include integers, fractions, terminating and nonterminating decimals, and irrational numbers.

For example,

$$-3, \quad 0, \quad 5, \quad \frac{1}{2}, \quad 2.75, \quad \sqrt{2}, \quad \pi$$

are all real numbers.

In Python, however, real numbers are not all represented in exactly the same way.

For example, a decimal number such as

2.75

is represented by Python using the data type `float`.

We can check this using the `type()` function:

```
type(2.75)
```

Python gives:

```
<class 'float'>
```

So we have another connection between mathematics and Python:

$$2.75 \rightarrow 2.75$$

and Python identifies 2.75 as a `float`.

Notice the difference:

```
type(5)
```

gives

```
<class 'int'>
```

while

```
type(5.0)
```

gives

```
<class 'float'>
```

Mathematically, (5) and (5.0) represent the same numerical value. However, Python stores them using different data types.

So remember:

$$5 \rightarrow \text{int}$$

while

$$5.0 \rightarrow \text{float.}$$

This distinction becomes important when we begin performing numerical calculations in Python.

iii. Complex Numbers In mathematics, a complex number is a number that can be written in the form

$$a + bi,$$

where (a) and (b) are real numbers and

$$i = \sqrt{-1}.$$

For example,

$$3 + 2i, \quad -1 + 4i, \quad 5 - 3i$$

are complex numbers.

A complex number has two parts:

$$a + bi$$

where (a) is called the **real part** and (b) is called the **imaginary part**.

Python also supports complex numbers. However, Python uses the letter j instead of the mathematical (i).

For example, the mathematical number

$$3 + 2i$$

is written in Python as:

```
3 + 2j
```

Python represents complex numbers using the data type `complex`.

We can check this using:

```
type(3 + 2j)
```

which gives:

```
<class 'complex'>
```

So we now have three important connections:

Integer $\rightarrow$ `int`

Decimal number $\rightarrow$ `float`

Complex number $\rightarrow$ `complex`

For example:

```
type(5)
```

gives `int`,

```
type(5.0)
```

gives `float`, and

```
type(5 + 2j)
```

gives `complex`.

These are some of the basic numerical data types we will use throughout this course.

2. Arithmetic $\rightarrow$ Python Arithmetic

Now that we understand how Python represents different types of numbers, the next step is to see how we perform **arithmetic operations** in Python.

In mathematics, we perform operations such as addition, subtraction, multiplication, and division.

Python can perform these same operations.

Addition

In mathematics, we write

$$2 + 3 = 5.$$

In Python, we use the + symbol for addition:

```
2 + 3
```

The result is

```
5
```

So,

$$2 + 3 \longrightarrow 2 + 3.$$

Subtraction

In mathematics,

$$7 - 4 = 3.$$

In Python, we use the - symbol:

```
7 - 4
```

The result is

```
3
```

Multiplication

In mathematics, we usually write

$$4 \times 5 = 20.$$

But in Python, we do not use the multiplication sign ($\times$). Instead, we use the asterisk *.

So we write:

```
4 * 5
```

The result is

```
20
```

Therefore,

$$4 \times 5 \longrightarrow 4 * 5.$$

Division

In mathematics,

$$10 \div 2 = 5.$$

In Python, we use the forward slash / for division:

```
10 / 2
```

The result is

```
5.0
```

Notice something interesting here. Mathematically, the answer is simply

$$5.$$

But Python gives us 5.0.

Why?

Because Python's / operator performs **true division**, and the result is represented as a **float**.

We will come back to this when we discuss Python's division operators in more detail.

2.1 Exponentiation → Python **

In mathematics, we use exponents to represent repeated multiplication.

For example,

$$2^3 = 2 \times 2 \times 2 = 8.$$

In Python, we use two asterisks ** for exponentiation:

```
2 ** 3
```

The result is:

```
8
```

So the mathematical expression

$$2^3$$

is written in Python as:

```
2 ** 3
```

Another example:

$$5^2 = 25.$$

In Python:

```
5 ** 2
```

gives:

```
25
```

Notice that Python does **not** use ^ for exponentiation. We use **.

So,

$$a^n \longrightarrow a ** n$$

For example,

$$3^4 \rightarrow 3 ** 4.$$

This is an important translation to remember because the symbol used in mathematics is different from the symbol used in Python.

2.2 Floor Division and Remainder → Python Arithmetic

In mathematics, when we divide one integer by another, the result may be a whole number or it may have a remainder.

For example,

$$17 \div 5 = 3 \text{ remainder } 2.$$

This means

$$17 = 5(3) + 2.$$

Python provides two operators that allow us to work directly with these parts of a division.

Floor Division

In Python, we use `//` for **floor division**.

For example:

```
17 // 5
```

gives:

```
3
```

The `//` operator gives the quotient after division, rounded down to the next integer.

For positive integers, this is the same as the whole number part of the division.

For example,

```
20 // 4
```

gives:

```
5
```

because

$$20 \div 4 = 5.$$

Remainder

Python uses the `%` operator to find the **remainder** after division.

For example:

```
17 % 5
```

gives:

```
2
```

because

$$17 = 5(3) + 2.$$

So,

```
17 // 5
```

gives the quotient:

```
3
```

while

```
17 % 5
```

gives the remainder:

```
2
```

Together, these operators allow us to express the division

$$17 = 5(3) + 2$$

in Python.

We can also verify this:

```
5 * (17 // 5) + (17 % 5)
```

which gives:

```
17
```

So we can think of the operators this way:

Mathematical idea	Python
Quotient	//
Remainder	%

These operators become especially useful when we work with **whole number division, counting, patterns, and sequences**.

2.3 Order of Operations → Python

When we perform several arithmetic operations in the same mathematical expression, we follow a particular order.

For example,

$$2 + 3 \times 4$$

does **not** mean

$$(2 + 3) \times 4.$$

According to the usual order of operations, we perform multiplication before addition:

$$2 + 3 \times 4 = 2 + 12 = 14.$$

Python follows the same basic order of operations.

For example:

```
2 + 3 * 4
```

gives:

```
14
```

Python performs the multiplication first and then the addition.

Using Parentheses

Just as in mathematics, we can use parentheses to tell Python which operation we want to perform first.

For example,

$$(2 + 3) \times 4 = 5 \times 4 = 20.$$

In Python:

```
(2 + 3) * 4
```

gives:

```
20
```

Compare the two expressions:

```
2 + 3 * 4
```

gives:

```
14
```

while

```
(2 + 3) * 4
```

gives:

```
20
```

The parentheses change the order in which the operations are performed.

A Mathematical Expression in Python

Consider the mathematical expression

$$\frac{3 + 5}{2}.$$

In Python, we write:

```
(3 + 5) / 2
```

which gives:

4.0

Another example is

$$2^3 + 4 \times 5.$$

In Python:

```
2 ** 3 + 4 * 5
```

Python evaluates the exponent first, then the multiplication, and finally the addition:

$$2^3 + 4 \times 5 = 8 + 20 = 28.$$

Therefore:

```
2 ** 3 + 4 * 5
```

gives:

```
28
```

The important idea here is that **Python follows rules for evaluating mathematical expressions**, just as we do in mathematics.

So when we translate mathematics into Python, we must pay attention not only to the symbols we use, but also to the **structure of the mathematical expression**.

Summary: Mathematics $\rightarrow$ Python Arithmetic

Mathematical idea	Python
+	+
−	−
$\times$	*
$\div$	/
a^n	a ** n
Quotient	//
Remainder	%

A mathematical expression can be represented as a Python expression.

Mathematical expression $\rightarrow$ Python expression $\rightarrow$ Python evaluates the expression